파이썬 전처리 및 시각화

강의 개요

1 **Numpy, Pandas:** 데이터 효율 처리

2 **Matplotlib, Seaborn:** 인사이트 시각화

학습 환경 설정

패키지 설치

```
# 가상환경 생성 및 활성화
conda create -n dataviz python=3.10
conda activate dataviz

# 필요 패키지 설치
pip install numpy pandas matplotlib seaborn
pip install jupyter scikit-learn
```

Numpy: 빠른 숫자 계산 도구

Numpy: 파이썬 고속 숫자 계산 도구. 데이터 효율적 처리.

다차원 배열

ndarray로 데이터 고속 계산, 메모리 효율적 사용.

수학 계산

다양한 수학 함수 제공.

고속 처리

C언어 기반, 파이썬 속도 한계 극복.

Numpy: 데이터 과학 핵심. Pandas, Matplotlib 등 다른 분석 도구 기반.

Numpy 고급 배열 생성

다양한 배열 생성 함수

```
# 균등 간격 배열
np.linspace(0, 1, 5)  # [0., 0.25, 0.5, 0.75, 1.]

# 로그 스케일 배열
np.logspace(0, 3, 4)  # [1., 10., 100., 1000.]

# 랜덤 배열
np.random.rand(3, 3)  # 0-1 균등분포
np.random.randn(3, 3)  # 정규분포
```

특수 행렬 생성

```
# 단위 행렬
np.eye(3)

# 대각 행렬
np.diag([1, 2, 3])

# 상수 행렬
np.ones((2, 3))
np.zeros((2, 3))
np.full((2, 2), 7)
```

적절한 배열 생성 함수는 코드를 간결하게 유지하고 효율적인 데이터 구조를 생성하는 데 활용.

Numpy 브로드캐스팅과 고급 인덱싱

브로드캐스팅 규칙

다른 shape 배열 간 연산 지원

```
a = np.array([1, 2, 3])
b = np.array([[10], [20], [30]])
print(a + b)  # 자동 확장
# [[ 11  12  13]
#  [ 21  22  23]
#  [ 31  32  33]]
```

고급 인덱싱

복잡한 조건으로 데이터 필터링

```
arr = np.arange(10)
mask = (arr > 3) & (arr < 8)
print(arr[mask]) # [4 5 6 7]

# 정교한 마스킹
arr = np.random.randn(5, 5)
mask = (arr > 0) & (arr < 1)
arr[mask] = 0 # 조건에 맞는 값만 0으로 변경
```

Numpy 벡터화 연산과 성능 최적화

벡터화 중요성

```
# 비효율적인 방법 (루프 사용)
def slow_function(data):
    result = np.zeros_like(data)
    for i in range(len(data)):
        result[i] = data[i] ** 2 + 5
    return result

# 벡터화된 방법
def fast_function(data):
    return data ** 2 + 5

# 성능 차이는 수십~수백 배
```

벡터화 연산은 대용량 데이터 처리 필수 최적화 기법

내부적으로 C 코드로 실행되어 Python 루프보다 훨씬 빠름

함수	용도	간단 예시
`np.where(cond, x, y)`	조건부 선택	`np.where(a > 0, a, 0)`
`np.select(condlist, choicelist, default)`	다중 조건부 선택	`np.select([a<0, a==0], ['neg', 'zero'], 'pos')`
`np.vectorize(func)`	스칼라 함수 배열화	`np.vectorize(lambda x: x**2)(arr)`
`np.einsum(subscripts, *operands)`	아인슈타인 표기법 (행렬·벡터 연산)	`'ij,j->i'`
`np.choose(indices, choices)`	인덱스 기반 선택	`np.choose([0,2,1], [[1,2,3],[4,5,6],[7,8,9]])`
`np.piecewise(x, condlist, funclist)`	구간별 함수 적용	`np.piecewise(x, [x<0, x>=0], [lambda x: -x, lambda x: x])`

함수	용도	간단 예시
`np.fromfunction(func, shape)`	좌표 기반 배열 생성	`np.fromfunction(lambda i,j: i+j, (3,3))`
`np.apply_along_axis(func, axis, arr)`	축별 사용자 함수 적용	`np.apply_along_axis(np.sum, 1, arr)`
`np.broadcast_to(array, shape)`	명시적 브로드캐스팅	`np.broadcast_to([1,2], (3,2))`
`np.tile(array, reps)`	배열 반복	`np.tile([1,2], (3,1))`
`np.repeat(array, repeats, axis)`	원소 반복	`np.repeat([1,2], 3)`
`np.meshgrid` / `np.mgrid`	2D/3D 좌표 그리드 생성	`x,y = np.meshgrid([1,2,3],[4,5])`
`np.unique`	고유 값 추출	`np.unique([1,2,2,3])`
`np.isin(element, test_elements)`	포함 여부 검사	`np.isin([1,2,3], [2,4])`

실습: 간단한 색상 필터 적용

NumPy 활용, 이미지 특정 색상 영역 조작 필터 구현

01

특정 색상 범위 (RGB/HSV) 선택

이미지 색상 공간 RGB/HSV 변환

02

해당 영역 색상 변경

정의된 색상 범위 내 픽셀 찾아 색상 변경

03

결과 이미지 저장

수정된 NumPy 배열 이미지 형식으로 저장 및 원본과 비교

Pandas: 데이터 분석의 강력한 도구

설계자: Wes McKinney

2008년, Wes McKinney는 금융 데이터 분석 고성능 파이썬 라이브러리 부재 해결 위해 Pandas 개발

설계 배경: 현실 문제 해결

파이썬에 데이터프레임 구조 도입, 데이터 정렬, 결측치 및 시계열 처리 기능 등 효율적 구현

1

직관성과 생산성

최소한 코드로 직관적, 유연한 API 제공.

2

일관된 데이터 모델

Index, Series, DataFrame 등 일관된 모델로 다양한 데이터 형태 처리.

3

파이썬 생태계 통합

NumPy 기반 성능, SciPy, Matplotlib 등과 원활한 연동.

4

실전 데이터 기능

시계열, 범주형 데이터, 결측값 처리 등 현업 기능 내장.

5

오픈소스 및 커뮤니티 주도

대규모 커뮤니티 지속 발전, 사용자 피드백 반영.

Pandas: 직관적인 생산성

Pandas는 최소 코드로 복잡한 데이터 작업을 효율적으로 처리하는 강력한 생산성 도구.

그룹별 데이터 집계

```
import pandas as pd

df = pd.DataFrame({
    'Team': ['A', 'A', 'B', 'B'],
    'Score': [10, 20, 30, 40]
})

df.groupby('Team')['Score'].mean()
```

결과: 몇 줄로 그룹별 통계 계산 가능, 데이터 탐색 시간 단축

날짜/시간 데이터 처리

```
df = pd.read_csv('sales.csv',
                 parse_dates=['Date'],
                 index_col='Date')
df['Sales'].resample('M').sum()
```

결과: CSV 파일 읽기 시 날짜 데이터 즉시 처리, 월별 매출 손쉽게 집계

일관된 데이터 모델

Series와 DataFrame에서 동일 API

```
s = pd.Series([1, 2, 3])
df = pd.DataFrame({'A': [1, 2, 3]})

print(s.mean())   # 2.0
print(df.mean())  # A    2.0
```

동일 인덱스 기반 연산

```
s1 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
s2 = pd.Series([10, 20, 30], index=['b', 'c', 'a'])

s1 + s2
```

파이썬 생태계와의 통합

1

NumPy 연산과 혼합 사용

```
import numpy as np
import pandas as pd

df = pd.DataFrame(np.random.randn(3, 3),
columns=list('ABC'))
df['sum'] = np.sum(df, axis=1)
```

Pandas DataFrame, NumPy 배열과 완벽 호환

2

Matplotlib과 시각화 통합

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.DataFrame({'Sales': [100, 120, 150]}, index=['Jan',
'Feb', 'Mar'])
df['Sales'].plot(kind='line', title='Monthly Sales')
plt.show()
```

실전 데이터 기능

결측치 처리

```
df.fillna(0)      # 결측치 0으로 채우기
df.dropna()       # 결측치 행/열 제거
```

시계열 데이터 슬라이싱

```
df['2024-01':'2024-03'] # 24년 1월-3월 데이터 선택
```

Pandas: 데이터 분석의 강력한 도구

데이터프레임

2차원 테이블 형태 데이터 구조

데이터 필터링

조건에 맞는 데이터 추출

데이터 변환

reshape, pivot, melt 등 구조 변경

데이터 결합

다양한 join/merge 방식

Pandas는 SQL과 스프레드시트 장점을 결합한 파이썬 라이브러리. 데이터 분석 핵심 도구.

Pandas 인덱스와 다중 인덱스

인덱스의 중요성

Pandas 인덱스, 단순 행 번호 넘어선 데이터의 핵심 식별자

```
# 인덱스 설정 및 재설정
df = df.set_index('customer_id')
df = df.reset_index()

# 인덱스 기반 선택
df.loc['C001']  # 라벨 기반
df.iloc[0]      # 위치 기반
```

다중 인덱스(MultiIndex)

```
# 다중 인덱스 생성
df = df.set_index(['region', 'date'])

# 다중 인덱스 접근
df.loc[('서울', '2023-01-01')]

# 인덱스 레벨 다루기
df.index.get_level_values(0)
df.swaplevel(0, 1)
```

Pandas 데이터 병합 고급 기법

데이터 병합 (Merge)

```
import pandas as pd


df1 = pd.DataFrame({'key1': ['A', 'B', 'C'], 'key2': [1, 2, 3], 'val1': [10, 20, 30]})
df2 = pd.DataFrame({'key1': ['B', 'C', 'D'], 'key2': [2, 3, 4], 'val2': [100, 200, 300]})

# Inner Join (기본값): 일치하는 키만 결합
inner_merged_df = pd.merge(df1, df2, on=['key1', 'key2'], how='inner')
print("Inner Join:\n", inner_merged_df)
```

```
# 서로 다른 키 이름 병합 (left_on, right_on)
df_a = pd.DataFrame({'id_a': ['X', 'Y'], 'data_a': [5, 6]})
df_b = pd.DataFrame({'id_b': ['Y', 'Z'], 'data_b': [7, 8]})
diff_key_merged_df = pd.merge(df_a, df_b, left_on='id_a', right_on='id_b', how='inner')
print("\nDifferent Key Names Merge:\n", diff_key_merged_df)
```

Inner Join 결과

key1	key2	val1	val2
B	2	20	100
C	3	30	200

Different Key Names Merge 결과

id_a	data_a	id_b	data_b
Y	6	Y	7

데이터 연결 (Concatenation)

수직 연결 (행 추가)

```
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})

pd.concat([df1, df2], axis=0)
```

결과

A	B
1	3
2	4
5	7
6	8

수평 연결 (열 추가)

```
df3 = pd.DataFrame({'C': [9, 10]})
pd.concat([df1, df3], axis=1)
```

결과

A	B	C
1	3	9
2	4	10

Pandas GroupBy

1. 기본 그룹화 및 집계

```
import pandas as pd

df = pd.DataFrame({
    'Team': ['A', 'A', 'B', 'B', 'C'],
    'Points': [10, 15, 20, 25, 30]
})

df.groupby('Team')['Points'].mean()
```

결과

Team	Points
A	12.5
B	22.5
C	30.0

2. 여러 컬럼 그룹화

```
import pandas as pd

df = pd.DataFrame({
    'Team': ['A', 'A', 'B', 'B', 'C', 'C'],
    'Sport': ['Football', 'Basketball', 'Football', 'Basketball', 'Football',
'Basketball'],
    'Points': [10, 15, 20, 25, 30, 35]
})

df.groupby(['Team', 'Sport'])['Points'].sum()
```

결과

Team	Sport	Points
A	Football	10
	Basketball	15
B	Football	20
	Basketball	25
C	Football	30
	Basketball	35

3. 여러 집계 함수 적용

Team	Points
A	12.5
B	22.5
C	30.0

```
df.groupby('Team')['Points'].agg(['mean', 'max', 'min'])
```

결과

Team	mean	max	min
A	12.5	15	10
B	22.5	25	20
C	30.0	30	30

4. 집계 시 컬럼명 변경

```
df.groupby('Team').agg(
    avg_points=('Points', 'mean'),
    max_points=('Points', 'max')
)
```

결과

Team	avg_points	max_points
A	12.5	15
B	22.5	25
C	30.0	30

결측치 처리 전략

데이터셋에서 값이 누락된 상태(NaN)를 의미.

결측치는 분석 및 모델 학습 시 오류나 편향을 유발할 수 있으므로 적절한 처리가 필요하다.

예시 데이터 및 확인

```
df = pd.DataFrame({
    'A': [1, 2, np.nan, 4],
    'B': [4, 5, 6, np.nan],
    'C': [7, np.nan, 9, 10]
})
```

행 삭제 (dropna())

```
df_drop_row = df.dropna()
print(df_drop_row)
```

결과:

1.0	4.0	7.0

모든 컬럼에 값이 있는 첫 번째 행만 남는다.

열 삭제 (dropna(axis=1))

```
df_drop_col = df.dropna(axis=1)
print(df_drop_col)
```

결과:

1.0	4.0
2.0	5.0
NaN	6.0
4.0	NaN

데이터 정규화와 표준화

Min-Max 정규화

데이터 0-1 범위 변환

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df[['col']] = scaler.fit_transform(
    df[['col']]
)
```

Z-점수 표준화

평균 0, 표준편차 1 변환

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df[['col']] = scaler.fit_transform(
    df[['col']]
)
```

Robust 스케일링

이상치에 강건함

```
from sklearn.preprocessing import RobustScaler
scaler = RobustScaler()
df[['col']] = scaler.fit_transform(
    df[['col']]
)
```

문자열 데이터 처리와 정규식

Pandas 문자열 메서드

```
# 기본 문자열 처리
df['text'] = df['text'].str.lower()
df['text'] = df['text'].str.strip()

# 문자열 결합 및 분할
df['full_name'] = df['first'].str.cat(
    df['last'], sep=' '
)
df[['city', 'country']] = df['location'].str.split(
    ',', expand=True
)

# 포함 여부 확인
mask = df['text'].str.contains('키워드')
```

정규식을 활용한 고급 처리

```
# 패턴 매칭 및 대체
df['clean_text'] = df['text'].str.replace(
 r'[^가-힣a-zA-Z0-9]', '', regex=True
)

# 패턴 추출
df['emails'] = df['text'].str.extract(
 r'([a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+)'
)

# 복잡한 패턴 처리
df['has_phone'] = df['text'].str.match(
 r'\d{3}-\d{4}-\d{4}'
)
```

카테고리 데이터 인코딩

원-핫 인코딩

```
df = pd.DataFrame({
    'id': [1, 2, 3, 4],
    'category': ['A', 'B', 'A', 'C']
})

# 원-핫 인코딩
dummies = pd.get_dummies(
    df, columns=['category'],
    prefix='cat'
)
print(dummies)
```

결과

id	cat_A	cat_B	cat_C
1	1	0	0
2	0	1	0
3	1	0	0
4	0	0	1

레이블 인코딩

```
df = pd.DataFrame({
    'id': [1, 2, 3, 4],
    'category': ['A', 'B', 'A', 'C']
})

# 레이블 인코딩
df['category_code'] = df['category'].astype('category').cat.codes
print(df)
```

결과

id	category	category_code
1	A	0
2	B	1
3	A	0
4	C	2

날짜/시간 데이터 처리

시계열 데이터에서 날짜/시간 컴포넌트는 중요한 피처임. 시간적 패턴을 포착하는 피처 생성.

1. 날짜 변환

```
import pandas as pd

df = pd.DataFrame({
    'date': ['2025-08-01', '2025-08-15', '2025-08-16']
})

# 문자열 → 날짜 변환
df['date'] = pd.to_datetime(df['date'])
```

결과

date
2025-08-01
2025-08-15
2025-08-16

2. 날짜 속성 추출

```
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
```

결과

date	year	month
2025-08-01	2025	8
2025-08-15	2025	8
2025-08-16	2025	8

3. 날짜 간격 계산

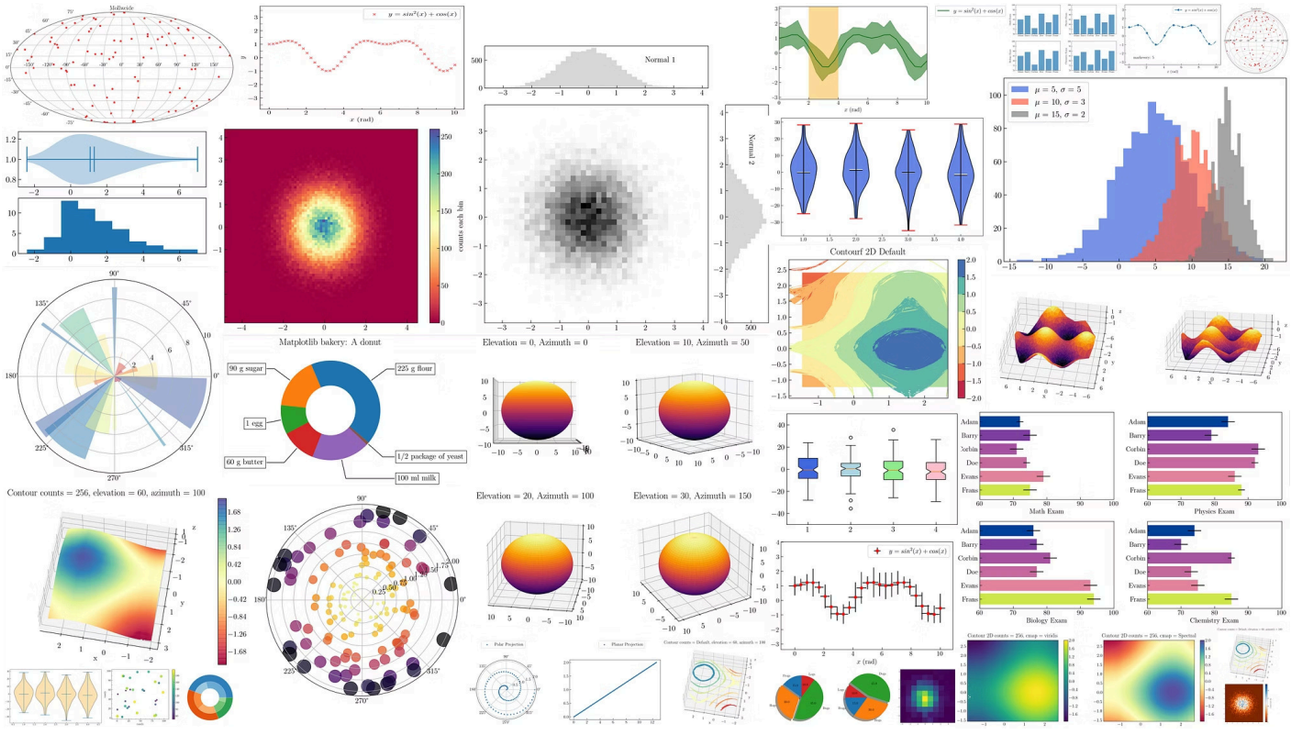
```
df['start_date'] = pd.to_datetime(['2025-08-01', '2025-08-10', '2025-08-12'])
df['end_date']   = pd.to_datetime(['2025-08-05', '2025-08-15', '2025-08-20'])

df['days_diff'] = (df['end_date'] - df['start_date']).dt.days
```

결과

start_date	end_date	days_diff
2025-08-01	2025-08-05	4
2025-08-10	2025-08-15	5
2025-08-12	2025-08-20	8

Matplotlib: 데이터 시각화의 기본



구조 이해

Figure(전체 그림)와 Axes(개별 서브플롯)로 구성된 객체 지향적 API

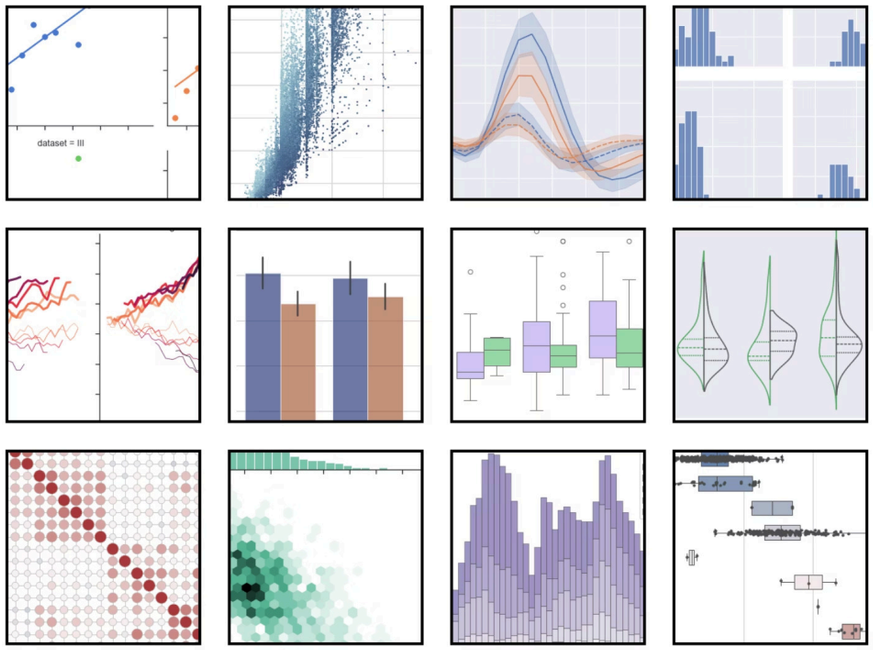
```
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(x, y)
ax.set_title('제목')
ax.set_xlabel('x축 레이블')
```

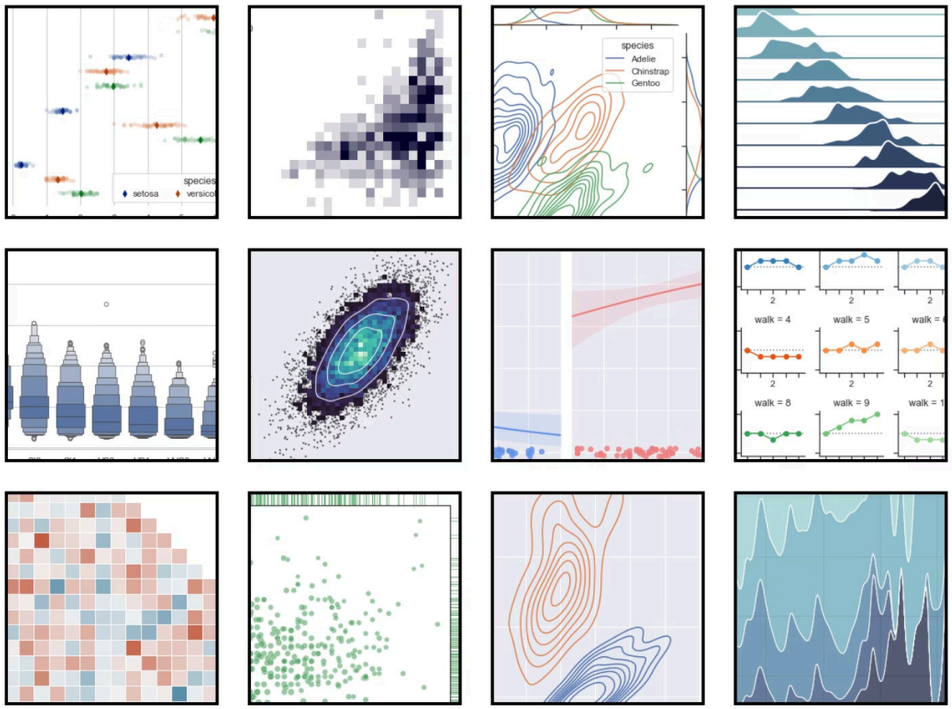
스타일링

색상, 마커, 선 스타일, 폰트 등 세부 커스터마이징

```
plt.style.use('seaborn-whitegrid')
ax.plot(x, y, color='#1c9770',
        linestyle='--', marker='o',
        linewidth=2, markersize=8)
```

Seaborn: 통계 시각화





실습 : 온라인 리테일 매출 분석

01

데이터 생성 (NumPy)

- 2023년 온라인 리테일 샘플 데이터 생성 (NumPy 난수 기능 활용)
- 필드 포함: 주문 ID, 날짜, 고객 ID, 상품 카테고리, 단가, 수량, 프로모션 여부

02

데이터 정제·가공 (Pandas)

- 생성 데이터 정제 및 가공 (현업 데이터처럼 다듬기)
- 가격 데이터 결측치 처리
- 데이터 집계: 일별/카테고리별 매출, 고객별 구매 패턴 등
- 파생 변수 생성: 매출액, 월, 주말 여부

03

데이터 시각화 (Matplotlib / Seaborn)

- 정제·가공 데이터 시각화 (Matplotlib/Seaborn 활용)
- 시각화 내용: 월별 매출 추세, 매출액 분포, 카테고리별 총 매출, 프로모션별 매출 비교
- 목표: 데이터 패턴 시각적 파악

04

인사이트 도출

- 시각화 분석 기반 비즈니스 인사이트 도출
- 매출 영향 요인, 효과적 프로모션 타겟, 매출 급증 시기